# INGENIERÍA DE SEGURIDAD US

**E.T.S. Ingeniería Informática. Universidad de Sevilla**
Avda. Reina Mercedes s/n. 41012 Sevilla
**Tlf/Fax** 954 557 139 **E-mail** lsi@lsi.us.es **Web** www.lsi.us.es


## PAI 4. DevSecOps - ASEGURAMIENTO DE LA CADENA DE SUMINISTRO EN EL DESARROLLO DE APLICACIONES WEB

### *Introducción*

Una entidad de desarrollo de aplicaciones Web, que es cliente de INSEGUS ha experimentado recientemente varios incidentes de seguridad debido a vulnerabilidades en sus sistemas debido a tráfico malicioso en la red. Para mejorar su **postura de seguridad**, la empresa ha decidido implementar un enfoque ***DevSecOps*** utilizando diferentes herramientas de automatización. Actualmente la empresa sigue para sus desarrollos un Pipeline de para integración y despliegue continuo (CI/CD) como el de la Figura 1.

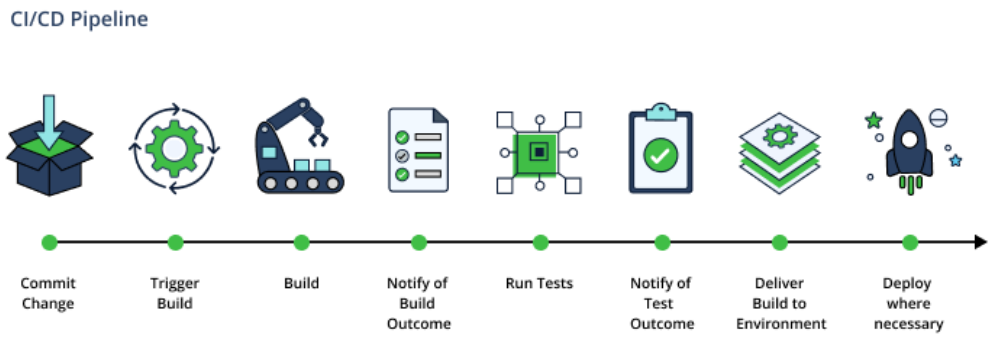


**Figura 1: Pipeline CI/CD**

La cuestión que se plantea el responsable es si los incidentes de seguridad eran debido a esta forma de trabajar y en la forma de comprobar (realización de test) el cumplimiento de las Políticas de Seguridad empresariales establecidas.

Además, en dicha entidad se complica el escenario para llevar a cabo los tests de seguridad, si pensamos que el desarrollo se está pasando al escenario de aplicaciones software monolíticas a aplicaciones basadas en ***microservicios*** **(Figura 2)**, con el consiguiente aumento de la superficie de ataque.

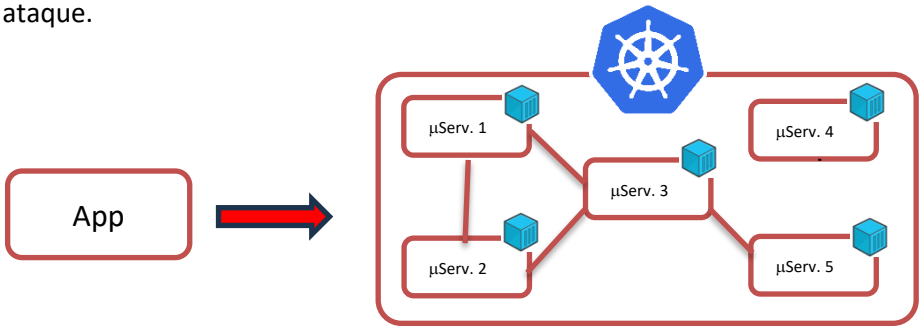


**Figura 2:** Arquitectura microservicio de app

Y además estos microservicios están siendo ejecutados en ***Containers***, por lo tanto, mucho más aumento de la superficie de ataque, a lo que se añade las plataformas de nubes públicas que

usan y además de utilizar ***Kubernetes*** para administrar las aplicaciones en contenedores, que añade aún mucha más superficie de ataque y por tanto debemos tener que atender a muchos más ciberriesgos.

Dado la gran cantidad de participantes en cualquier desarrollo. Todo ello ha llevado a la empresa plantearse acudir a la automatización de lo que serían las actividades de análisis y mitigación de dichos ciberriesgos durante el desarrollo y despliegue en producción. Por tanto, dado el proceso de DevOps automatizado que es llevado a cabo actualmente en la empresa requiere también que los test de seguridad se hagan de forma automatizada a lo largo de todo el Pipeline del ciclo de desarrollo, la propuesta que nos hacen es usar un esquema **DevSecOps** que consistiría según dichos cliente, en el esquema que se muestra en la Figura 3.

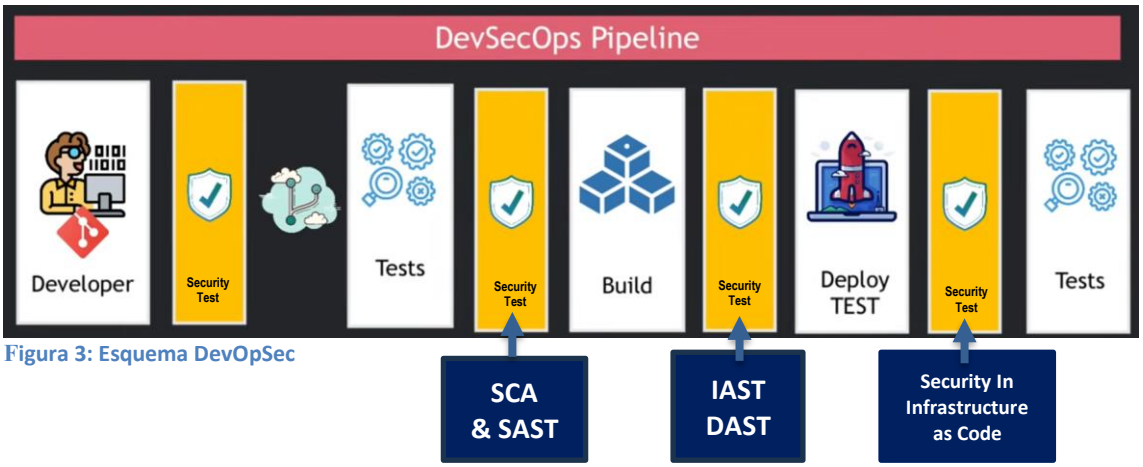


Figura 3: Esquema DevOpSec

Las herramientas que consideran se podrían ser **Source Composition Analysis (SCA), Static application security testing** (**SAST), Dynamic application security testing DAST y Security In Infrastructure as Code**. Estas herramientas se podrían integrar para su gestión en un único sistema, como podría ser *DefectDojo*, para facilitar toda la gestión de las vulnerabilidades y facilitar una ***Security as Code***, encontrar los bugs de seguridad en etapas más tempranas, por tanto, más rapidez en los ciclos de liberación de código seguro, además se evita detectar y corregir bugs en producción que son mucho más costosos.

## *Objetivos del proyecto*

Por todo lo anterior, lo que se le propone a los Security Teams es **poner en marcha un Pipeline DevSecOps de la forma lo más automatizada posible que permita la entrega del producto software funcional y seguro** y que incluiría los siguientes objetivos:

1. **Definir una Pipeline CI/CD en un proyecto de prueba** en un repositorio de código (SCM).
2. Seleccionar al menos **tres herramientas de Testeo de Seguridad (SCA, SAST, IAST, DAST, Secuity IaC) para incluirlas en el Pipeline para dotarlos de capacidades DevSecOps.**
3. **Integrar las herramientas de Testeo** en el ciclo de vida del desarrollo para hacer Pipeline DevSecOps en un ciclo de integración continua.
4. **Desarrollar los tests correspondientes que permitan detectar vulnerabilidades en cada etapa** del Pipeline DevSecOp.
5. **Seleccionar** una herramienta de gestión de vulnerabilidades para **integrar, priorizar y clasificar las vulnerabilidades de Seguridad que se vayan encontrando por las herramientas de testeo en el Pipeline DevSecOps**. Como recomendación podría usarse *DefectDojo* o cualquier otra herramienta que se considere oportuna.

## Recomendaciones

### *Pipelines CI/CD*

- **GitLab CI/CD:** https://docs.gitlab.com/ci/
- **GitHub CI/CD:** https://github.com/resources/articles/devops/ci-cd
- **Jenkins:** https://www.jenkins.io/
  **Documentation Jenkins**: https://www.jenkins.io/doc/

### *Source Composition Analysis (SCA)*

- **Depex**: https://github.com/GermanMT/depex
- **GitHub Dependabot:** https://github.com/dependabot
- **Snyk**: https://snyk.io/
- **OWASP Dependency-Check**: https://owasp.org/www-project-dependency-check/
- **Sonatype Nexus Lifecycle:** https://www.sonatype.com/products/open-source-security-dependency-management

### *SAST*

- **Catálogo de herramientas de análisis de código estático:** https://owasp.org/www-community/Source_Code_Analysis_Tools
- **SonarQube**: https://www.sonarsource.com/open-source-editions/sonarqube-community-edition/

### *IAST*

- **Semgrep:** https://github.com/semgrep/semgrep
- **RISP (para PHP):** https://github.com/robocoder/rips-scanner
- **Contrast Security (pago)**
- **Seeker (pago)**

### *DAST*

- **Catálogo de herramientas:** https://owasp.org/www-community/Free_for_Open_Source_Application_Security_Tools
- **OWASP ZAP:** https://www.zaproxy.org/
- **Burp Suite:** https://portswigger.net/burp/communitydownload
- **Wapiti:** https://wapiti-scanner.github.io/

### *Security in Infrastructure as Code (IaC)*

Detecta errores de configuración, vulnerabilidades y malas prácticas en archivos como Terraform, CloudFormation, Kubernetes YAML, Dockerfiles y Ansible.

- **KICS**: https://kics.io/index.html
- **Trivy**: https://trivy.dev/latest/
- **Terrascan** https://github.com/tenable/terrascan

### *Gestión de vulnerabilidades*

- **DefectDojo:** https://defectdojo.com/community
- **Documentación de Defect Dojo:** https://docs.defectdojo.com/
  **Proyecto prueba**: https://github.com/DefectDojo/django-DefectDojo
  **Integración Jenkins:** https://plugins.jenkins.io/defectdojo/
  **Integración GitLab:** https://gitlab.com/to-be-continuous/defectdojo/

### *Postproducción - Vulnerabilidades en aplicaciones desplegadas*

**Configuraciones inseguras y servicios vulnerables**

- **OpenVAS**: https://www.openvas.org/
- **Tenable Nessus**: https://www.tenable.com/downloads
- **Nexpose** (pago con prueba gratuita): https://www.rapid7.com/products/nexpose/

## Normas del entregable

- El plazo de entrega de dicho proyecto finaliza el **día 27 de abril a las 23:59 horas.**
- Cada ***Security Team*** debe entregar a través de la Plataforma de Enseñanza Virtual y en la **actividad** preparada para ello un archivo zip, nombrado ***PAI4-ST<Num>*.zip**, que deberá contener al menos los ficheros siguientes:
  - **Código fuente de las posibles implementaciones y/o scripts desarrollados y/o configuraciones establecidas en herramientas ya disponibles. Así como de los tests y logs con todas las evidencias de las pruebas realizadas.**
  - **Documento PDF (máximo 10 páginas), estructura del documento:**
    - Informe técnico donde se describan las herramientas seleccionadas, el pipeline CI/CD seleccionado, y la integración realizada.
    - Manual de despliegue y uso con instrucciones de instalación y puesta en funcionamiento del proyecto.
    - Grado de completitud donde se indicará de manera trazable el grado de cumplimiento de los objetivos.
- Los proyectos entregados fuera del plazo establecidos serán considerados inadecuados por el cliente y por tanto entrarán en penalización por cada día de retraso entrega de 10% del total, hasta agotarse los puntos.
- **El cliente no se aceptará envíos realizados por email, ni mensajes internos de la enseñanza virtual, ni correo interno de la enseñanza virtual. Toda entrega realizada por estos medios conllevará una penalización en la entrega del 10%.**